Java Backend Architecture

Interview Series

Chapter 8.7
API Security

50+ Interview Q&A | Code Examples | Architecture Diagrams | Advanced Topics

Chapter 8.7 — API Security

Introduction

API security encompasses multiple complementary mechanisms: API keys for client identification, request signing for integrity and authentication, rate limiting to prevent abuse and DoS, mTLS for service-to-service authentication, and API gateway enforcement for centralised policy. Senior engineers must understand when to apply each mechanism and how they compose together.

API Key Authentication

API Key: hash(key) stored in DB (SHA-256, never plaintext). Key prefix (sk_live_) for identification, suffix for DB lookup.

Rate limit: token bucket per API key. Gateway enforces. 429 Too Many Requests on exceed.

Rotate keys: issue new key, deprecation period, revoke old. Separate keys per environment (dev/staging/prod).

```
// Spring Security — API key authentication filter @Component public class ApiKeyAuthFilter extends OncePerRequestFilter { private final ApiKeyService apiKeyService; @Override protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain chain) throws ServletException, IOException { String apiKey = request.getHeader("X-API-Key"); if (apiKey == null) { chain.doFilter(request, response); return; } // Hash key for lookup (never store raw key) String keyHash = sha256Hex(apiKey); ApiKeyDetails details = apiKeyService.findByHash(keyHash) .orElseThrow(() -> new BadCredentialsException("Invalid API key")); UsernamePasswordAuthenticationToken auth = new UsernamePasswordAuthenticationToken( details.getClientId(), null, details.getAuthorities()); SecurityContextHolder.getContext().setAuthentication(auth); chain.doFilter(request, response); } private String sha256Hex(String input) { return Hashing.sha256().hashString(input, StandardCharsets.UTF_8).toString(); } }
```

Request Signing (HMAC-SHA256)

1. Canonical string: METHOD+URL+BODY_HASH+TIMESTAMP+NONCE
2. signature = HMAC-SHA256(canonical_string, secret_key)
3. Headers: X-Timestamp, X-Nonce, X-Signature
4. Reject if |now - X-Timestamp| > 5 minutes (replay protection)
5. Reject if X-Nonce seen before (nonce cache in Redis)
6. Recompute signature, compare with X-Signature (constant-time)

Constant-time comparison prevents timing attacks: MessageDigest.isEqual() — not String.equals().

AWS Signature V4 uses this pattern: canonical request + string-to-sign + HMAC signing key hierarchy.

```
// Java: HMAC-SHA256 request signing (client-side) public class RequestSigner { public Map<String, String> sign(String method, String url, String body, String secretKey) throws Exception { String timestamp = String.valueOf(Instant.now().getEpochSecond()); String nonce = UUID.randomUUID().toString(); String bodyHash = sha256Base64(body != null ? body : ""); String canonical = method + "\n" + url + "\n" + bodyHash + "\n" + timestamp + "\n" + nonce; String signature = hmacSha256Base64(canonical, secretKey); return Map.of( "X-Timestamp", timestamp, "X-Nonce", nonce, "X-Signature", signature ); } private String hmacSha256Base64(String data, String key) throws Exception { Mac mac = Mac.getInstance("HmacSHA256"); mac.init(new
```

```
SecretKeySpec(key.getBytes(StandardCharsets.UTF_8), "HmacSHA256")); return
Base64.getEncoder().encodeToString(mac.doFinal(data.getBytes(StandardCharsets.UTF_8))); } } //
Server-side: verify (constant-time comparison!) boolean valid = MessageDigest.isEqual(
expected.getBytes(StandardCharsets.UTF_8), received.getBytes(StandardCharsets.UTF_8));
```

Rate Limiting

Rate limit headers: X-RateLimit-Limit: 100, X-RateLimit-Remaining: 45, X-RateLimit-Reset: 1700000060

429 Too Many Requests + Retry-After header when limit exceeded.

Bucket4j + Redis: distributed rate limiting. Bucket.asScheduler().consume(1) non-blocking; tryConsume(1) returns boolean.

```
// Bucket4j + Redis rate limiting in Spring Boot @Component public class RateLimitFilter
extends OncePerRequestFilter { private final ProxyManager<String> proxyManager; // Bucket4j
Redis proxy @Override protected void doFilterInternal(HttpServletRequest req,
HttpServletResponse res, FilterChain chain) throws ServletException, IOException { String key =
resolveKey(req); // e.g., API key or IP Bucket bucket = proxyManager.builder() .build(key,
this::newBucketConfig); ConsumptionProbe probe = bucket.tryConsumeAndReturnRemaining(1);
res.setHeader("X-RateLimit-Remaining", String.valueOf(probe.getRemainingTokens())); if
(probe.isConsumed()) { chain.doFilter(req, res); } else { long retryAfter =
probe.getNanosToWaitForRefill() / 1_000_000_000; res.setHeader("Retry-After",
String.valueOf(retryAfter)); res.sendError(429, "Too Many Requests"); } } private
BucketConfiguration newBucketConfig() { return BucketConfiguration.builder()
.addLimit(Bandwidth.classic(100, Refill.greedy(100, Duration.ofMinutes(1)))) .build(); } }
```

API Gateway Security

Gateway	Auth Feature	Rate Limiting	Config
Kong	key-auth, jwt, oauth2 plugins	rate-limiting plugin (local/Redis)	Declarative (YAML/deck) or Admin API
AWS API Gateway	Lambda authorizer, Cognito, IAM	Usage plans + API keys	CloudFormation / CDK / Console
Spring Cloud Gateway	TokenRelay filter, JWT filter	RequestRateLimiter (Redis)	application.yaml route predicates
NGINX Plus	auth_jwt module, auth_request	limit_req_zone directive	nginx.conf

```
# Spring Cloud Gateway — rate limiting + JWT validation spring: cloud: gateway: routes: - id:
orders-service uri: lb://orders-service predicates: - Path=/api/orders/** filters: -
TokenRelay= # Forward access token to downstream - name: RequestRateLimiter args:
redis-rate-limiter.replenishRate: 10 redis-rate-limiter.burstCapacity: 20 key-resolver:
"#{@apiKeyResolver}"
```

mTLS for Service-to-Service

In zero-trust architectures, every service-to-service call must be mutually authenticated. mTLS (Mutual TLS) provides this: both the calling service and the receiving service present certificates. In a service mesh (Istio, Linkerd), mTLS is enforced automatically between pods. Without a mesh, configure Spring Boot with client-auth: need and provision certificates via cert-manager.

AWS WAF + API Gateway

Protection	AWS WAF Rule Type	What It Blocks

SQL Injection	AWSManagedRulesSQLiRuleSet	SQL injection patterns in request
XSS	AWSManagedRulesCommonRuleSet	Cross-site scripting payloads
IP Reputation	AWSManagedRulesAmazonIpReputationList	Known bad IPs, botnets
Rate-based rules	Custom rate-based rule	More than N requests per 5 min from IP
Geo blocking	Geo match statement	Traffic from specific countries
Bot Control	AWSManagedRulesBotControlRuleSet	Automated bot traffic

50+ Interview Questions & Answers

Q1. What is API key authentication?

A client identification mechanism where each client is issued a unique secret key. Sent in a request header (X-API-Key) or query parameter. Server hashes the received key and compares with stored hash — never store plaintext keys.

Q2. How do you securely store API keys?

Never store plaintext. Store SHA-256 hash of the key. Show the full key only once (at generation). Format: prefix_randomBase62 — prefix (sk_live_, sk_test_) identifies the environment; hash the full key for DB lookup.

Q3. What is the difference between API key and OAuth2 access token?

API key: long-lived, identifies a client application (not a user). OAuth2 access token: short-lived, represents delegated user permission, scoped, revocable. API keys are simpler but less secure (no expiry, harder to scope finely).

Q4. What is request signing?

A security mechanism where the client creates a signature (HMAC-SHA256) of the request content (method, URL, body hash, timestamp, nonce) using a shared secret. Server recomputes and verifies — ensures integrity and authenticity.

Q5. Why use request signing instead of just API keys?

API key only proves client identity — a replay of a captured request will succeed. Request signing adds: timestamp (replay window), nonce (unique per request), body hash (integrity). Stolen signed request cannot be replayed after the time window.

Q6. What is a replay attack?

Attacker captures a valid HTTP request and resends it. Prevention: include timestamp (reject requests older than 5 minutes) + nonce (unique per request, stored in Redis for the validity window). Both together prevent replay attacks.

Q7. What is constant-time comparison and why is it needed?

Comparing two strings character-by-character with early exit leaks timing information (how many characters match). An attacker can use timing differences to guess secrets. MessageDigest.isEqual() compares all bytes regardless — same time regardless of where they differ.

Q8. What is rate limiting?

Controlling the number of requests a client can make in a time window. Prevents abuse, DoS attacks, brute force. Returns 429 Too Many Requests with Retry-After header when limit exceeded.

Q9. What is the token bucket algorithm?

A bucket holds tokens (capacity=burst). Each request consumes one token. Tokens are added at a fixed rate. If bucket is empty, reject request. Allows bursting up to capacity, then throttles to the refill rate. Bucket4j implements this.

Q10. What is the sliding window log algorithm?

Store a timestamp for each request in a sorted set (Redis ZSET). On each request: remove entries older than window, count remaining, reject if over limit, add new timestamp. More accurate than fixed window but uses more memory.

Q11. What is the difference between token bucket and leaky bucket?

Token bucket: allows bursting (unused capacity accumulates as tokens). Leaky bucket: constant output rate regardless of bursts — excess is queued or dropped. Token bucket is more common for API rate limiting.

Q12. What is Bucket4j?

A Java rate-limiting library implementing token bucket algorithm. Supports: in-memory (single instance), Hazelcast, Redis (via Redisson), Caffeine. Integrates with Spring Boot. Provides tryConsume (boolean) and consumeIgnoringRateLimits methods.

Q13. What headers should rate-limit responses include?

X-RateLimit-Limit: total limit. X-RateLimit-Remaining: remaining in window. X-RateLimit-Reset: Unix timestamp when limit resets. Retry-After: seconds to wait (on 429). Follow IETF draft RateLimit header fields.

Q14. What is mTLS and when to use it for APIs?

Mutual TLS — both client and server present certificates. Use for service-to-service communication in zero-trust architectures, high-security B2B APIs (banking, healthcare), and IoT device authentication. Stronger than API keys.

Q15. What is an API gateway?

A reverse proxy that acts as the entry point for API traffic. Provides: authentication, authorization, rate limiting, SSL termination, request transformation, logging, analytics. Examples: Kong, AWS API Gateway, Spring Cloud Gateway, NGINX.

Q16. What is the API key rotation strategy?

Issue new key with overlap period. Client migrates to new key. After migration period, deprecate old key (warn in response headers). Then revoke. Maintain audit log of all key events. Automate via key management service.

Q17. What is AWS WAF?

Web Application Firewall that inspects HTTP requests before they reach your application. Rules: managed rule groups (SQL injection, XSS, bot control, known bad IPs), custom rules (rate-based, geo-match, size-based). Attaches to API Gateway, CloudFront, ALB.

Q18. What is a Lambda authorizer in AWS API Gateway?

A Lambda function that validates incoming tokens or API keys. Returns an IAM policy (Allow/Deny) for the request. Two types: token-based (receives auth header value) and request-based (receives full request context). Result can be cached.

Q19. What is Kong?

Open-source API gateway built on NGINX + OpenResty (Lua). Plugin ecosystem: key-auth, jwt, oauth2, rate-limiting, request-size-limiting, cors, bot-detection, ACL. Declarative config (deck/YAML), Admin API, or DB-backed.

Q20. What is the difference between authentication and authorization at the API gateway?

Authentication: verify identity (validate JWT signature, check API key). Authorization: check permissions (scope, role, resource ownership). Both can be enforced at the gateway, but fine-grained authorization often needs service-level context.

Q21. What is CORS and how is it related to API security?

Cross-Origin Resource Sharing — controls which origins can call your API from a browser. Not a security measure (browsers enforce it, not servers). Configure CORS headers correctly — avoid Access-Control-Allow-Origin: *. Spring: @CrossOrigin or CorsConfigurationSource.

Q22. What is an HMAC-based API key?

Instead of using a random opaque key, the key is an HMAC of (client_id, timestamp, secret). Allows the server to validate the key without a DB lookup (if the key structure is known). Amazon AWS uses this approach for its presigned URLs.

Q23. What is a presigned URL?

A URL containing an embedded signature that grants temporary access to a resource without requiring the caller to have credentials. AWS S3 presigned URLs: HMAC of canonical request + expiry. Used for temporary file upload/download access.

Q24. What is API versioning from a security perspective?

Old API versions with known vulnerabilities should be deprecated and removed. Enforce version header/path. Monitor deprecated version usage. Set sunset dates. Security patches should be applied to all supported versions simultaneously.

Q25. What is a circuit breaker and how does it relate to API security?

Circuit breaker prevents cascading failures — when downstream service fails, fast-fail instead of waiting. From security: prevents resource exhaustion attacks where slow/failing upstream services cause thread pool exhaustion.

Q26. What is API schema validation?

Validate incoming request bodies/parameters against an OpenAPI/JSON Schema specification. Reject requests that don't conform — prevent injection attacks with malformed data. Spring Boot: use Hibernate Validator or explicit OpenAPI validation filter.

Q27. What is content-type validation?

Verify the Content-Type header matches the expected format (application/json). Reject requests with unexpected content types — prevents content-type confusion attacks. Spring: configure supported media types in RequestMapping.

Q28. What is request size limiting?

Reject requests exceeding a maximum body size. Prevents memory exhaustion DoS attacks. Spring Boot: spring.servlet.multipart.max-file-size and max-request-size. Kong: request-size-limiting plugin. AWS API Gateway: 10MB max payload.

Q29. What is input sanitization vs validation?

Validation: reject invalid input (fail fast). Sanitization: clean/normalize input (e.g., strip HTML, trim whitespace). Prefer validation — sanitization can miss attack vectors. Use allowlists (what is permitted) not denylists (what is blocked).

Q30. What is the Retry-After header?

HTTP response header indicating how long to wait before making another request. Required with 429 Too Many Requests. Value: seconds (integer) or HTTP date. Clients that respect it implement exponential backoff. Spring: set in RateLimitFilter response.

Q31. What is API key prefix (e.g., sk_live_, pk_)?

A short prefix prepended to the API key. Purpose: identify key type/environment at a glance, enable secret scanning (GitHub can detect sk_live_ keys in code commits), and DB lookup (store prefix+hash, use prefix to find the right key hash). Popularized by Stripe.

Q32. What is adaptive rate limiting?

Dynamically adjusting rate limits based on server load, user behavior, or time of day. Example: reduce limits during high load, increase for trusted clients, temporary burst allowance for known batch jobs. More complex than fixed limits.

Q33. What is IP-based rate limiting vs key-based?

IP-based: rate limit per client IP — simple, but shared IPs (corporate NAT, VPN) cause false positives. Key-based: rate limit per API key — more accurate, but requires authentication. Best practice: both — IP for unauthenticated endpoints, key for authenticated.

Q34. What is distributed rate limiting?

Rate limiting across multiple service instances using a shared store (Redis). Each instance checks and updates Redis atomically (Lua script or Redis EVAL). Bucket4j with Redis proxy, Redis sorted sets (sliding window), or Redis counters with EXPIRE.

Q35. What is the OAuth2 token introspection approach for API security?

Resource server calls /introspect with each incoming token. Pros: real-time revocation. Cons: latency, load on auth server. Alternative: JWT (validate locally) + short expiry + revocation list cache. Hybrid: JWT for speed + occasional introspection for sensitive ops.

Q36. What is idempotency key for API security?

A client-generated unique ID per request (Idempotency-Key header). Server stores the key and response. If the same key arrives again (retry), return the cached response — prevent duplicate operations (double charge). Different from nonce (used for replay attack prevention in signing).

Q37. How do you secure WebSocket APIs?

Authenticate at upgrade time (HTTP headers in the upgrade request). Pass JWT in query param or subprotocol header. Spring Security: configure WebSocket security with AbstractSecurityWebSocketMessageBrokerConfigurer. Rate limit message frequency.

Q38. What is GraphQL API security?

GraphQL-specific concerns: query depth limiting (prevent deep nested queries DoS), query complexity analysis (cost per field), field-level authorization, introspection disabling in production, persisted queries. Spring for GraphQL: @PreAuthorize on resolver methods.

Q39. What is the OWASP API Security Top 10?

1. BOLA/IDOR — broken object level auth. 2. Broken Authentication. 3. BOPLA — broken object property level auth. 4. Unrestricted Resource Consumption (rate limiting). 5. BFLA — broken function level auth. 6. Unrestricted access to sensitive business flows. 7. SSRF. 8. Security misconfiguration. 9. Improper inventory management. 10. Unsafe consumption of APIs.

Q40. What is BOLA (Broken Object Level Authorization)?

API endpoint accepts user-controlled object IDs without verifying the requester owns the object. GET /orders/12345 — if any authenticated user can access any order ID, that's BOLA. Fix: always verify the authenticated user is the owner of the requested resource.

Q41. What is mutual authentication vs one-way TLS?

One-way TLS: only server presents certificate (standard HTTPS). Mutual (mTLS): both server and client present certificates. Mutual authentication verifies both parties' identities. Required when you can't rely on other auth mechanisms (API key, JWT).

Q42. What is a JWT bearer token attack?

Attacker: 1) None algorithm attack (set alg=none), 2) RS256→HS256 attack (change to HS256, sign with server's public key as HMAC secret), 3) kid injection (inject SQL/path traversal in kid claim). Defense: whitelist algorithms, validate kid against known set.

Q43. What is API security testing?

OWASP ZAP (automated scanning), Burp Suite (manual + automated), Postman (functional + security), openapi-fuzzer (fuzzing based on schema), OWASP API Security Testing Guide. Test: auth bypass, injection, IDOR, excessive data exposure, mass assignment.

Q44. What is excessive data exposure?

API returns more data than the client needs — e.g., full user object when only name is needed. Front-end filters the display but the data is in transit. Fix: design minimal response DTOs, use projections, review every field returned.

Q45. What is mass assignment vulnerability?

Client sends more fields than intended in a request body, and the server blindly applies all of them (e.g., setting isAdmin=true in a user update). Fix: use explicit DTOs, @JsonIgnore sensitive fields, or Jackson's DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES.

Q46. What is API gateway vs service mesh for security?

API gateway: north-south traffic (external client to services). Service mesh: east-west traffic (service-to-service). Both needed: gateway for external auth/rate limit, mesh (Istio) for internal mTLS + authorization policies. Not either/or.

Q47. How do you implement API key scoping?

Associate each API key with allowed scopes/permissions in the DB. When key is used, load its scopes and set as GrantedAuthority in the Authentication. @PreAuthorize checks apply. Example: key A can only call read endpoints, key B can call write endpoints.

Q48. What is a service account vs user account for API access?

Service account: represents a non-human entity (application/service). Long-lived, no MFA. Use OAuth2 client_credentials grant (no user). User account: represents a human. Short-lived sessions, MFA. Use OAuth2 auth code flow. Never use human credentials for service-to-service.

Q49. What is the security risk of API keys in URLs?

API keys in URL query parameters appear in: server logs, browser history, referrer headers, CDN logs. Always use request headers (X-API-Key). Never log full request URLs without scrubbing. HTTPS does not protect query parameters from being logged.

Q50. What is certificate pinning for API clients?

Mobile/desktop apps pin the server certificate or public key. If a MITM presents a different cert (even CA-signed), the connection is rejected. Implemented in OkHttp via CertificatePinner. Requires careful key

rotation planning to avoid breaking clients.